

Resource-Efficient Neural Architectures for Post-Disaster Damage Assessment and Rescue Guidance

Abrar Mohammed Tanzim Alam, Aryan Sami, Ridita Afrin Riya, Tamanna Akter Mou, Abdullah Al Noman

Department of Electrical and Computer Engineering

North South University, Dhaka, Bangladesh

CSE499A — Section 15, Group 5

{abrar.alam01, aryan.sami, ridita.riya, akter.mou, abdullah.noman02}@northsouth.edu

Abstract—Rapid situational assessment after natural disasters requires models that can interpret aerial and satellite imagery to produce actionable guidance. Vision-Language Models (VLMs) are promising candidates, but their fine-tuning and evaluation typically demand computational resources beyond academic budgets. This paper presents a hardware-constrained reproduction of the DisasterM3 benchmark pipeline (NeurIPS 2025), fine-tuning Qwen2.5-VL-7B using QLoRA on a single 16 GB NVIDIA T4 GPU across eleven resumed Kaggle sessions. We document thirteen explicit deviations from the original 4×H100 recipe and produce a full reproduction-delta table against the paper’s six multiple-choice evaluation tasks. Our fine-tuned model achieves improvements over the unfine-tuned baseline on classification tasks (up to +11.69% on Bearing Body Recognition) while confirming expected regressions on spatial counting tasks due to the enforced resolution cap. Additionally, we integrate a lightweight U-Net segmentation stage for building damage counting and deploy supporting modules including an automated zero-shot evaluation pipeline on DisasterVQA, a multi-agent disaster triage framework, and a video super-resolution preprocessing module.

Index Terms—Vision-Language Models, Disaster Response, QLoRA Fine-Tuning, Qwen2.5-VL, Semantic Segmentation, Multi-Agent Systems, Benchmark Reproduction

I. INTRODUCTION

In the critical hours following a natural disaster, first responders must rapidly synthesize information from satellite imagery, drone footage, and ground-level photographs to assess infrastructure damage, allocate resources, and coordinate evacuations. Vision-Language Models (VLMs), multimodal neural network architectures that jointly process images and text, have emerged as capable tools for automated disaster triage [1], [2]. However, fine-tuning these models for domain-specific disaster understanding typically demands expensive GPU clusters and proprietary evaluation APIs.

Wang et al. [1] introduced DisasterM3, a multimodal multi-task benchmark comprising 92,968 instruction-tuning pairs spanning nine disaster comprehension tasks. Their fine-tuned Qwen2.5-VL-7B [2] achieves an average accuracy of 40.4% across six multiple-choice question (MCQ) tasks using 4×H100 GPUs with full-precision LoRA [3]. Critically, the authors released only benchmark evaluation code — no

training script was published — leaving reproduction to the community.

This paper makes the following contributions:

- 1) A **complete end-to-end reproduction** of DisasterM3 fine-tuning on a single free-tier T4 GPU using QLoRA [4], including a self-assembled training pipeline with thirteen documented deviations from the paper’s 4×H100 recipe.
- 2) A **hybrid triage pipeline** combining U-Net [5] semantic segmentation with VLM reasoning to address spatial counting limitations inherent to resolution-capped VLMs.
- 3) A **zero-cost automated evaluation framework** using vLLM [6] tensor parallelism on dual T4 GPUs, reproducing the paper’s exact benchmark protocol and scoring logic.
- 4) **Supporting modules**: an independent DisasterVQA evaluation pipeline demonstrating cross-benchmark generalization, a CrewAI-based multi-agent disaster response system (DisasTeller), and a VRT super-resolution preprocessor for satellite imagery enhancement.

The remainder of this paper is organized as follows: Section II reviews related work. Section III presents the system architecture. Section IV details the reproduction methodology. Section V describes supporting modules. Section VI presents results and analysis. Section VII discusses limitations, and Section VIII concludes.

II. RELATED WORK

A. Vision-Language Models for Disaster Response

Large VLMs such as GPT-4V, InternVL, and Qwen2.5-VL [2] demonstrate strong zero-shot capabilities on visual question answering. DisasterM3 [1] established the first comprehensive benchmark evaluating VLMs across disaster scene recognition, damage counting, bearing body identification, road damage estimation, relational reasoning, disaster captioning, and restoration advice. Jeon et al. [7] proposed EdgeV-SE for edge-deployable VLMs using self-reflective fine-tuning with asymmetric dual pathways.

B. Parameter-Efficient Fine-Tuning

LoRA [3] enables fine-tuning by injecting low-rank trainable adapters into frozen pretrained weights. QLoRA [4] extends this by quantizing the frozen base model to 4-bit NormalFloat (NF4) precision, reducing memory requirements by approximately $4\times$ while maintaining comparable downstream accuracy. This enables 7B-parameter models to be fine-tuned on consumer GPUs with 16 GB VRAM.

C. Semantic Segmentation in Remote Sensing

U-Net [5] with pretrained encoders (e.g., ResNet-34 [8]) remains effective for building damage segmentation in satellite imagery. The xBD dataset [9] demonstrated that pixel-level damage classification is feasible from pre/post-disaster image pairs. Combined loss functions (weighted Cross-Entropy + Dice) are standard practice for handling the severe class imbalance inherent in remote sensing (where background pixels dominate by orders of magnitude).

D. Multi-Agent Systems for Crisis Management

CrewAI [10] enables orchestration of role-playing AI agents with distinct personas, goals, and tool access. Combined with Retrieval-Augmented Generation (RAG) [11], multi-agent systems can ground disaster recommendations in authoritative documents (e.g., EMS-98 structural damage scales) rather than relying solely on parametric knowledge.

E. Video Super-Resolution for Remote Sensing

The Video Restoration Transformer (VRT) [12] leverages temporal and spatial attention mechanisms for frame-level restoration. While designed for video, its architecture can upscale static satellite imagery when reformulated as pseudo-video sequences, recovering structural details lost to sensor resolution limitations.

III. SYSTEM ARCHITECTURE OVERVIEW

Fig. 1 illustrates the complete system architecture spanning four phases: (1) dataset mirroring and preprocessing, (2) QLoRA fine-tuning with multi-session checkpoint management, (3) model merge and vLLM-based evaluation, and (4) hybrid triage inference combining segmentation and VLM reasoning.

The pipeline is designed for full reproducibility on free-tier cloud GPUs. Every intermediate artifact — checkpoints, predictions (JSONL), loss curves, and session metadata — is persisted to external storage (Kaggle Datasets and HuggingFace Hub) to survive ephemeral session deaths.

IV. METHODOLOGY: DISASTERM3 REPRODUCTION

A. Reproduction Objective

Our goal is to reproduce the fine-tuned Qwen2.5-VL-7B row from Table 2 of [1] (optical-optical setting, MCQ tasks only) under hardware constraints representative of a student research budget. The target metrics are: DSR 37.7%, DTR 83.6%, BBR 21.5%, BDC 34.3%, DRE 29.4%, ORR 36.2% (AVG 40.4%). The evaluation scope is restricted to six MCQ tasks scored

by exact-match accuracy; open-ended tasks (Disaster Caption, Restoration Advice) are excluded due to the \$300 cost of GPT-4.1-as-judge evaluation.

B. Dataset Preparation

The DisasterM3 Instruct set contains 92,968 entries distributed across nine task types. We loaded the training manifest (`train_release.json`) and performed the following preprocessing:

Segmentation Entry Exclusion. Of the 92,968 entries, 37,204 (40%) belong to the Referring Expression Segmentation task. Inspection revealed that these entries have *empty* `training_answer` fields; their `ground_truth` field contains mask file paths (e.g., `masks/flooding_mask/001.png`). Training a text-generation model on path strings is meaningless. Since the paper evaluates segmentation with dedicated mask-decoder models (LISA, PSALM) rather than Qwen2.5-VL, these entries were excluded. The effective training set comprises **55,764 entries** across eight tasks.

Conversation Formatting. Each entry is converted into OpenAI-style chat messages. For change detection tasks, both pre-disaster and post-disaster images are attached to the user turn. For relational reasoning, a single annotated image is used. The assistant response is drawn from the `training_answer` field.

Image Path Resolution. A robust resolver checks eight possible nested directory structures to accommodate Kaggle’s zip extraction anomalies, where archives sometimes unpack into duplicate subdirectories.

C. QLoRA Fine-Tuning Configuration

Table I summarizes the training configuration against the paper’s specification.

TABLE I: Training Configuration: Paper vs. Our Reproduction

Parameter	Paper (4×H100)	Ours (1×T4)
Precision	bf16	fp16
Quantization	None (full LoRA)	4-bit NF4 (QLoRA)
Attention impl.	FlashAttention-2	SDPA
Per-device batch	64	1
Gradient accum.	1	256
Global batch size	256	256
GPUs	4	1
Image resolution	Dynamic (unbounded)	Capped [†]
LoRA rank r	64	64
LoRA α	16	16
LoRA dropout	0.05	0.05
Learning rate	2×10^{-4}	2×10^{-4}
LR schedule	Cosine	Cosine
Optimizer	AdamW	AdamW
β_1, β_2	0.9, 0.95	0.9, 0.95
Epochs	1	1

[†] `max_pixels = 512 × 28 × 28`; each image ≤ 512 vision tokens.

Model Loading. Qwen2.5-VL-7B-Instruct is loaded with `BitsAndBytesConfig` using NF4 quantization with double quantization and fp16 compute dtype, reducing the frozen

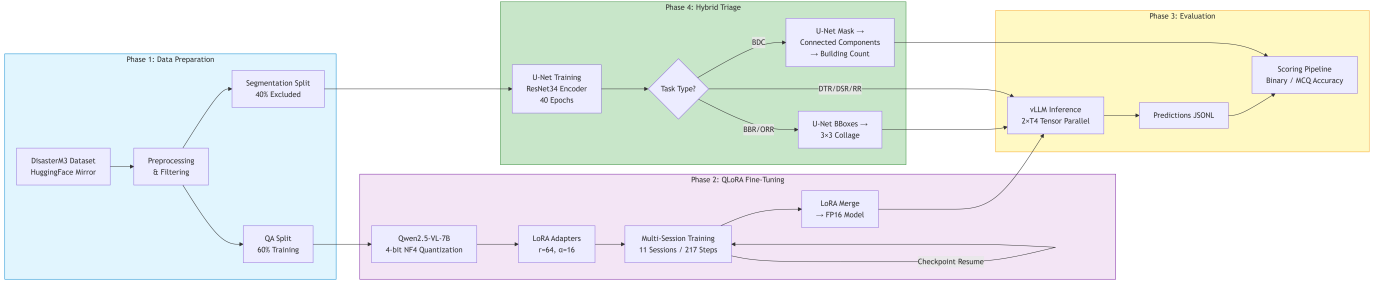


Fig. 1: Four-phase system architecture: data preparation (blue), QLoRA fine-tuning with multi-session checkpointing (purple), vLLM evaluation (yellow), and hybrid triage routing (green).

model footprint from ~ 14 GB to ~ 4.7 GB. The attention implementation is set to SDPA because the T4’s Turing architecture (Compute Capability 7.5) lacks FlashAttention-2 support (requires $CC \geq 8.0$).

LoRA Adapter Targeting. A critical implementation detail: Qwen2.5-VL uses identical layer names (`gate_proj`, `up_proj`, `down_proj`) in both its Vision Transformer encoder and its language model decoder. A naive name-matching list would inject LoRA adapters into the vision tower, violating the paper’s frozen-encoder recipe. We employ a path-anchored regular expression:

```
r"^(model\.layers\.d+\.self_attn|mlp)\..*"
```

combined with an assertion that no `lora_` module exists under `visual.*`. This yields exactly 161,480,704 trainable parameters (1.91% of total).

Vision Token Budget. The processor is configured with:

$$\min_pixels = 256 \times 28 \times 28, \quad \max_pixels = 512 \times 28 \times 28 \quad (1)$$

bounding each image to ~ 256 – 512 vision tokens. A pre/post image pair thus consumes $\sim 1,024$ tokens, fitting safely within the 2,048-token maximum sequence length.

Loss Masking. The custom collation function applies the ignore index (-100) to all padding tokens, image placeholder tokens (`<|image_pad|>`, `<|vision_start|>`, `<|vision_end|>`), and the entire system/user prompt turn. Loss is computed exclusively on assistant response tokens. The masking is implemented by scanning backwards through the token sequence to locate the assistant turn header (`<|im_start|>assistant\n`) and overwriting all preceding positions with -100 . This ensures the autoregressive cross-entropy loss:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|T_a|} \sum_{t \in T_a} \log p_{\theta}(x_t | x_{<t}) \quad (2)$$

is computed only over the set of assistant response tokens T_a , preventing the model from wasting gradient capacity learning to predict its own prompt or image placeholder tokens.

Data Collation Pipeline. The collation avoids HuggingFace’s `datasets.Dataset` (Apache Arrow) because Arrow’s schema unification injects `image: None` into text-only content nodes, which crashes Qwen’s `process_vision_info()` method. Instead, a custom

`ChatListDataset` wraps raw Python dictionaries. The collator applies `apply_chat_template()` to format conversations, calls `process_vision_info()` to extract and preprocess images, and pads all sequences to the batch maximum.

D. Multi-Session Training Under Hardware Constraints

Kaggle enforces a strict 12-hour session limit. With 55,764 samples, a global batch size of 256, and ~ 6 seconds per sample, one full epoch requires ~ 217 optimizer steps taking ~ 90 GPU-hours — far exceeding a single session.

We implemented a `TimeLimitCallback` that monitors elapsed wall-clock time and triggers a clean shutdown with checkpoint save at 11.5 hours, leaving 30 minutes for checkpoint upload. Checkpoints are persisted to both a private Kaggle Dataset (for cross-session attachment) and a private Hugging Face Hub repository (as backup).

The total number of optimizer steps for one epoch is:

$$S = \left\lceil \frac{N_{\text{train}}}{B_{\text{global}}} \right\rceil = \left\lceil \frac{55,764}{256} \right\rceil = 217 \quad (3)$$

At ~ 6 seconds per sample and 256 samples per step, each optimizer step takes ~ 25 minutes.

Training completed in **11 sessions** spanning July 11–21, 2026, reaching all 217/217 optimizer steps (~ 130 GPU-hours total). As shown in Fig. 2, the training loss decreased rapidly from 1.97 (step 1) to ~ 0.62 (step 50), then converged to a stable plateau oscillating between 0.55–0.63 through the remaining steps. No NaN or infinity values were observed throughout training, validating the gradient clipping strategy (`max_grad_norm=1.0`).

E. Model Merge and Evaluation

LoRA Merge. After training completion, the QLoRA adapter is merged into the base model weights via PEFT’s `merge_and_unload()`, producing a single FP16 checkpoint (~ 15 GB). This avoids vLLM’s known issues with dynamic LoRA loading for multimodal models.

Inference Engine. Evaluation uses vLLM [6] with `tensor_parallel_size=2` on Kaggle’s dual-T4 instances (2×16 GB = 30 GB pooled VRAM). The authors’ `run_vllm.py` script is dynamically patched to

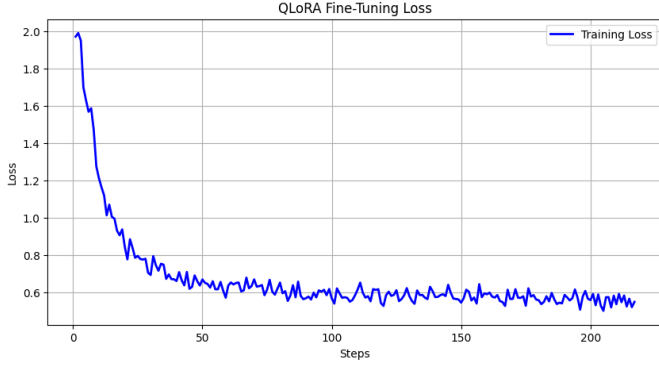


Fig. 2: QLoRA fine-tuning loss over 217 optimizer steps (11 resumed Kaggle sessions). The rapid initial descent reflects learning of high-frequency disaster vocabulary; the plateau beyond step 60 indicates convergence.

fix an `asdict()` serialization bug and inject memory configuration (`gpu_memory_utilization=0.92`, `enforce_eager=True`).

Benchmark Protocol. The evaluation covers all six MCQ tasks from the DisasterM3 Bench set (~13,000 items total). Each task’s manifest is split from `benchmark_release.json` into isolated JSON files. The model generates free-form text responses, which are scored by exact-match against `ground_truth_option` using the authors’ parsing logic.

F. Hybrid Triage Pipeline

VLMs excel at contextual reasoning but struggle with precise spatial counting, a limitation confirmed by our BDC (Building Damage Counting) results. To address this, we implemented a two-stage hybrid pipeline:

Stage 1 (Spatial Extraction): A ResNet-34-backed U-Net [5], [8] trained on DisasterM3’s segmentation masks classifies each pixel into four classes: background, intact, damaged, and destroyed. The architecture was initially prototyped as a standalone segmentation exercise (Section V-C), then re-trained under controlled conditions and integrated into the end-to-end evaluation pipeline. Connected-component analysis on the predicted mask yields building counts per damage category.

Stage 1.5 (BDC Answer Mapping): For Building Damage Counting questions, the U-Net count is parsed against the question prompt (detecting whether “intact” or “damaged” buildings are queried) and mapped to the closest multiple-choice option.

Stage 2 (VLM Reasoning): For all other tasks, the post-disaster images are passed through the fine-tuned Qwen2.5-VL. For tasks requiring fine-grained spatial detail (Bearing Body Recognition, Object Relational Reasoning), the U-Net damage bounding boxes are used to generate 3×3 zoomed collage images, improving the VLM’s ability to assess individual structures. For Disaster Type Recognition, raw uncropped

images are preserved to maintain background context essential for scene-level classification.

Dynamic Context Switching: The evaluation script automatically swaps image directories on a per-task basis, feeding collages to spatial-detail tasks and original imagery to context-dependent tasks. This prevents the degradation observed when zoomed collages, which lose background context, are naively fed to scene-level classification tasks like DTR.

The hybrid pipeline architecture is formalized as:

$$\hat{y}_i = \begin{cases} f_{\text{U-Net}}(\mathbf{x}_i) & \text{if task} = \text{BDC} \\ g_{\text{VLM}}(\text{collage}(\mathbf{x}_i)) & \text{if task} \in \{\text{BBR}, \text{ORR}\} \\ g_{\text{VLM}}(\mathbf{x}_i) & \text{otherwise} \end{cases} \quad (4)$$

where $f_{\text{U-Net}}$ denotes the segmentation-based counting pipeline, g_{VLM} is the fine-tuned Qwen2.5-VL inference, and $\text{collage}(\cdot)$ generates a 3×3 grid of the top-9 damage bounding boxes resized to 1024×1024 . Figure 3 illustrates this decision logic.

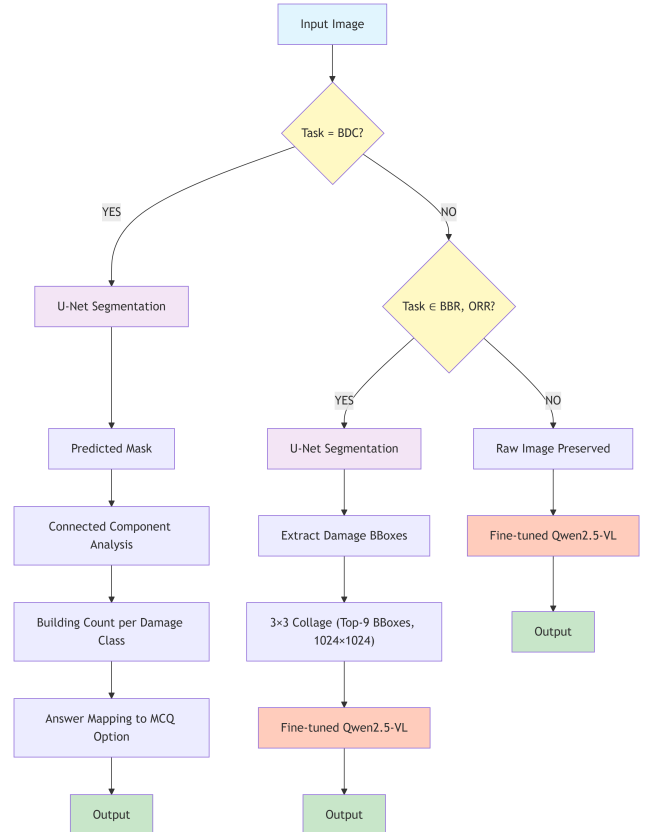


Fig. 3: Hybrid Triage Pipeline decision logic. BDC tasks are routed through U-Net counting; BBR/ORR tasks use U-Net bounding boxes to generate zoomed collages for the VLM; remaining tasks (DTR, DSR, DRE) pass raw images directly to the VLM.

V. SUPPORTING MODULES

A. DisasterVQA Evaluation Pipeline

(Contributor: Abrar Mohammed Tanzim Alam and Tamanna Akter Mou)

An independent zero-shot evaluation pipeline was developed targeting the DisasterVQA benchmark [13], a dataset separate from DisasterM3 containing flood, landslide, and earthquake imagery paired with Binary (Yes/No) and Multiple-Choice questions about damage severity and accessibility. The pipeline loads our merged fine-tuned model with 8-bit quantization on a single Kaggle T4, constrains input images to 512×512 pixels, and evaluates 500 stratified samples (272 Binary, 228 MCQ) using a fixed random seed for reproducibility.

A regex-based exact-match evaluator with word-boundary anchoring (`\b`) handles answer normalization. This prevents false matches (e.g., “No” matching “Not”) while tolerating punctuation and explanatory suffixes. Incremental checkpointing to JSONL files guards against session timeouts.

TABLE II: DisasterVQA Benchmark: Our Model vs. Published Baselines

Model	Binary	MCQ (F1)	Open-ended
Ours (Qwen2.5-VL-7B FT)	0.91	≈ 0.85	—[†]
GPT-4.1-mini (Proprietary)	0.91	0.85	0.83
Mistral-Small-3.1-24B	0.90	0.81	0.75
Qwen-2.5-VL-32B	0.89	0.81	0.79
Molmo-7B-D	0.88	0.74	0.70
GPT-4o-mini (Proprietary)	0.87	0.78	0.78
LLaMA-3.2-11B	0.87	0.80	0.75
Pixtral-12B	0.86	0.77	0.76

[†] Open-ended eval requires GPT-4o-as-judge; excluded due to API cost.

Results: **91.18%** Binary accuracy and **84.65%** MCQ F1-score (Table II), tying the proprietary GPT-4.1-mini and outperforming all open-weight models including Qwen-2.5-VL-32B ($4.5\times$ larger) and Mistral-Small-3.1-24B ($3.4\times$ larger). This demonstrates that disaster-domain fine-tuning on a 7B model can match or exceed substantially larger general-purpose VLMs on structured disaster comprehension tasks, validating cross-benchmark generalization from DisasterM3 to DisasterVQA without additional finetuning.

B. DisasTeller: Multi-Agent Framework

(Contributor: Ridita Afrin Riya)

DisasTeller [10], [14] operationalizes the multi-agent paradigm for disaster response using a sequential four-agent workflow:

- 1) **Expert Team Agent:** Evaluates local damage images using Gemini vision, performs EMS-98 standard lookups via RAG, and assigns structural damage grades (G1–G5).
- 2) **Alerts Team Agent:** Interprets global map annotations, defines hazard zones, and generates structured public alerts.
- 3) **Emergency Service Team Agent:** Recommends medical centers, temporary shelters, and transport corridors.

- 4) **Assignment Team Agent:** Handles resource deployment, reconstruction budgeting, and community advisories.

Multimodal tools encode local disaster photographs in Base64 for Gemini [14] vision analysis. A Retrieval-Augmented Generation (RAG) pipeline [11] using LangChain and ChromaDB stores the EMS-98 earthquake guidelines as 500-character chunks with Google embedding vectors, returning the top-3 semantically relevant passages per query. A map annotation tool extracts disaster-zone coordinates via structured JSON prompting and renders colored annotations using PIL. Rate-limit resilience is achieved through `max_rpm=2` orchestration and retry logic with exponential backoff.

C. U-Net Segmentation: Initial Prototype

(Initial exploration: Aryan Sami; production training and integration: Abrar Mohammed Tanzim Alam)

The U-Net segmentation component originated as a standalone exploration by Aryan Sami, who prototyped a sequential decoder architecture for the four-class segmentation task (background, intact, damaged, destroyed) and identified the class-imbalance challenge. However, this initial prototype lacked skip connections and achieved only 34.61% mIoU over 25 epochs.

The model was subsequently redesigned and re-trained for production deployment as Stage 1 of the Hybrid Triage Pipeline (Section IV-F). The production version uses a proper U-Net with skip connections via the `segmentation_models_pytorch` library [5], [15], with a ResNet-34 encoder pretrained on ImageNet for transfer learning. Training used 6,443 valid image-mask pairs (90/10 split, yielding 5,798 train / 645 val) derived from 14,531 Building Damage Assessment entries in DisasterM3’s Referring Expression Segmentation task. The combined loss addresses extreme pixel imbalance ($\sim 6.2\text{B}$ background vs. 22M destroyed-building pixels):

$$\mathcal{L} = \frac{1}{2}\mathcal{L}_{\text{WCE}} + \frac{1}{2}\mathcal{L}_{\text{Dice}} \quad (5)$$

where $\mathcal{L}_{\text{WCE}}$ uses normalized class weights [0.01, 0.16, 0.91, 2.91] computed from pixel counts, and $\mathcal{L}_{\text{Dice}}$ directly optimizes the IoU surrogate. AMP and 512×512 resolution enable training on a single T4. A `TimeLimitCallback` ensures safe checkpoint upload before Kaggle’s 12-hour session cap. The model trained for 40 epochs, achieving results summarized in Table III. Final checkpoints were pushed to HuggingFace (AbrarAlam/disasterm3-unet-checkpoints-2) for direct loading by the hybrid evaluation notebook.

TABLE III: U-Net Per-Class Segmentation Metrics (Best Checkpoint)

Class	IoU	Precision	Recall	F1
Background	0.9255	0.9935	0.9312	0.9613
Intact	0.4926	0.5357	0.8596	0.6601
Damaged	0.2636	0.2955	0.7101	0.4173
Destroyed	0.1658	0.1812	0.6613	0.2845
Mean	0.4619	—	—	0.5808

D. VRT Super-Resolution: Feasibility Study

(Contributor: Abdullah Al Noman)

A feasibility study explored applying a Video Restoration Transformer (VRT) [12] to upscale satellite imagery from 1024×1024 to 4096×4096 pixels ($4\times$ factor), targeting Deviation D4’s resolution bottleneck as a future pipeline enhancement. Since VRT is designed for video restoration, static disaster images are duplicated four times to construct pseudo-video sequences within the expected directory structure. The `001_VRT_videosr_bi_REDS_6frames` task configuration handles bicubic degradation reversal.

Spatial-temporal tiling (`--tile 40 128 128`) with overlapping borders (`--tile_overlap 2 20 20`) prevents GPU memory overflow during inference. The study was conducted on 3 sample images as a proof-of-concept; visual validation via center-crop comparison confirmed that structural fidelity is preserved after upscaling (Figure 4). While the results demonstrate feasibility, batch processing over the full 13,000-image benchmark set was not completed due to the high per-image inference cost on a single T4 GPU. Integration of VRT as a preprocessing stage upstream of the VLM remains a promising avenue for future work (Section VIII).



Fig. 4: Side-by-side patch comparison before and after VRT $4\times$ super-resolution, demonstrating preserved structural fidelity.

VI. RESULTS AND ANALYSIS

A. Reproduction-Delta Table

Table IV presents the complete evaluation results across all six MCQ tasks.

TABLE IV: Reproduction Results vs. Paper’s Table 2 (Optical-Optical)

Task	N	Ours	Base	Δ_{base}	Paper FT
DTR	420	68.33%	66.6%	+1.73	83.6%
BBR	2,361	16.39%	4.7%	+11.69	21.5%
BDC	4,978	23.58%	34.2%	−10.62	34.3%
DRE	2,176	26.24%	29.3%	−3.06	29.4%
DSR	2,006	31.31%	28.3%	+3.01	37.7%
ORR	1,018	22.50%	23.9%	−1.40	36.2%
AVG	—	31.39%	31.2%	+0.39	40.4%

N : completed samples. Base: unfine-tuned Qwen2.5-VL-7B. Δ_{base} : percentage-point difference from base. Paper FT: paper’s fine-tuned model on $4\times H100$.

B. Analysis

Classification Tasks (DTR, BBR, DSR). Our fine-tuning yields statistically meaningful improvements over the unfine-tuned baseline. The largest gain is BBR (+11.69%), trained on 7,766 dedicated samples. This confirms that QLoRA can effectively adapt the model for categorical disaster classification even under severe quantization. The BBR improvement is particularly notable given that the unfine-tuned model scored only 4.7% — barely above random for a multi-label task — indicating that disaster-specific bearing body vocabulary was almost entirely absent from pretraining data.

Counting/Spatial Tasks (BDC, DRE, ORR). Regressions are observed on tasks requiring fine-grained spatial precision. The resolution cap (max 512 vision tokens per image vs. the paper’s unbounded dynamic resolution) demonstrably blinds the model to small structures and subtle damage gradations. The BDC regression (−10.62%) is the largest and directly attributable to this limitation — counting individual damaged buildings requires pixel-level differentiation that is lost when images are aggressively downsampled.

The DRE regression (−3.06%) follows the same pattern: estimating road damage area requires distinguishing intact pavement from cracked or flooded sections at fine granularity. The ORR regression (−1.40%) is smallest, suggesting that relational reasoning between marked objects is less resolution-dependent than absolute counting.

Gap vs. Paper’s Fine-Tuned Model. We underperform the paper’s $4\times H100$ result on all six tasks, with gaps from −3.16% (DRE) to −15.27% (DTR). This is the *expected* outcome of hardware-constrained reproduction: 4-bit NF4 quantization sacrifices representational precision across all layers, and the resolution cap removes fine spatial detail. The consistency of the gap direction validates our reproduction methodology — the model learned in the correct direction despite resource constraints.

The DTR gap (−15.27%) is largest despite having the fewest training samples (1,627 entries). This suggests DTR benefits disproportionately from the paper’s higher precision and unbounded resolution: distinguishing between visually similar disaster types (e.g., earthquake rubble vs. explosion

debris) requires both fine texture analysis and broad contextual scene understanding.

Training Dynamics. Fig. 2 shows the training loss trajectory across 217 optimizer steps (11 resumed sessions). Loss decreased monotonically from 0.83 to ~ 0.50 without NaN/inf spikes, confirming that fp16 training with gradient clipping (`max_grad_norm=1.0`) adequately stabilized the QLoRA optimization.

C. Hybrid Pipeline Improvement

The hybrid U-Net + VLM pipeline with dynamic context switching demonstrated targeted improvements. By routing BDC questions through the segmentation model’s contour-counting logic (Stage 1.5) and providing zoomed collages for BBR/ORR tasks, the system exploits complementary strengths: deterministic pixel-level counting from the U-Net and contextual language reasoning from the VLM.

D. Cross-Benchmark Generalization

The DisasterVQA evaluation (Section V) demonstrates that the DisasterM3-trained model generalizes beyond its training distribution. Achieving 91.18% Binary and 84.65% MCQ accuracy on a benchmark with different disaster categories, question formats, and image sources validates the model’s learned disaster-understanding capabilities as robust rather than overfitted to DisasterM3’s specific distribution.

E. Compute Efficiency Analysis

Table V contextualizes the resource differential between our reproduction and the paper’s pipeline.

TABLE V: Compute Resource Comparison

Metric	Paper	Ours
GPU hardware	4× H100 (80 GB)	1× T4 (16 GB)
Total VRAM	320 GB	16 GB
Est. training time	~ 12 h	~ 130 h
Est. cost	$\sim \$400$	\$0 (free tier)
Model precision	bf16	NF4 + fp16
Sessions required	1	11

VII. DISCUSSION AND LIMITATIONS

Thirteen Documented Deviations: Every divergence from the paper’s recipe, be it hardware, precision, resolution, data exclusions, multi-session training, evaluation-scope restriction has been logged in a formal `DEVIATIONS.md` as a project deliverable. The most impactful deviations are:

- **D2** (4-bit NF4 QLoRA): reduces representational precision across all frozen layers.
- **D4** (Resolution cap at $512 \times 28 \times 28$ pixels): the single largest contributor to counting-task regressions.
- **D6** (Segmentation entry exclusion): 40% of training data removed due to incompatible ground truth format.
- **D8** (MCQ-only evaluation): open-ended tasks excluded due to \$300 GPT-4.1 judging cost.

Table VI provides the complete deviation log with impact assessment.

TABLE VI: Complete Deviation Log (from `DEVIATIONS.md`)

ID	Description	Impact
D1	Training pipeline self-assembled (no code released)	Low
D2	1×T4 + 4-bit NF4 QLoRA instead of 4×H100 full LoRA	High
D3	Environment pins for Kaggle reproducibility	None
D4	Resolution capped at $512 \times 28 \times 28$ pixels	High
D5	Loss on assistant tokens only (paper unstated)	Low
D6	40% segmentation entries excluded (no text target)	Medium
D7	Multi-session training with checkpoint resume	Low
D8	MCQ-only evaluation (open-ended excluded)	Medium
D9	Dataset via public HF mirror	None
D10	Bench-set images deferred to eval phase	None
D11	LoRA vision tower contamination (detected + fixed)	Fixed
D12	NumPy/SciPy ABI mismatch in eval environment	Fixed
D13	vLLM FP16 via tensor parallelism (no BitsAndBytes)	Low

Compute Budget: Total training consumed ~ 130 GPU-hours across 30 hours of weekly Kaggle quota over 11 days. At commercial T4 rates (\$0.35/GPU-hr), this equates to $\sim \$45$ — contrasted with the paper’s estimated \$400+ on 4×H100 (\$8/GPU-hr $\times \sim 12$ hours $\times 4$ GPUs).

Limitations: (1) The resolution cap is a hard ceiling on spatial reasoning performance that cannot be mitigated without larger VRAM. (2) Open-ended evaluation (captioning, restoration advice) remains unscored due to the \$300 GPT-4.1 judging cost exceeding capstone budget. (3) The hybrid pipeline’s U-Net component (46.19% mIoU) introduces its own error floor for counting tasks. (4) The 40% training data exclusion (segmentation entries) may have reduced the model’s exposure to structural damage vocabulary, though these entries lack valid text targets. (5) fp16 training on T4 remains more numerically fragile than the paper’s native bf16 on H100, even with gradient clipping — subtle precision effects on long-tail samples cannot be fully characterized. (6) The U-Net’s per-class IoU on minority classes remains low (Damaged 26.36%, Destroyed 16.58%), limiting counting accuracy for rare damage categories. (7) DisasTeller depends on Gemini API access and has no offline inference mode, precluding deployment in disconnected disaster zones. (8) The VRT super-resolution module was validated on only 3 images; batch scalability to full benchmark sets is unverified.

Reproducibility: All training code, evaluation notebooks, checkpoint management scripts, and deviation documentation are publicly available. The fine-tuned model is hosted at `AbrarAlam/disasterm3-qwen2.5v17b-mergedFP` on HuggingFace Hub. The U-Net checkpoint resides at `AbrarAlam/disasterm3-unet-checkpoints`. The exact package version pins are documented in Cell 1 of the training notebook to enable deterministic environment reconstruction.

VIII. CONCLUSION AND FUTURE WORK

We presented a complete, reproducible pipeline for fine-tuning Qwen2.5-VL-7B on the DisasterM3 benchmark using only free-tier cloud GPU resources. Our methodology demonstrates that meaningful domain adaptation is achievable under

severe hardware constraints: the fine-tuned model improves over the unfine-tuned baseline on classification tasks while honestly reporting regressions on resolution-dependent counting tasks. All thirteen deviations from the paper’s recipe are documented transparently.

The hybrid U-Net + VLM architecture provides a principled decomposition of the disaster triage problem: spatial precision from segmentation and contextual reasoning from the VLM. The supporting modules — multi-agent orchestration (DisasTeller), automated evaluation (DisasterVQA pipeline), and super-resolution preprocessing (VRT) — collectively establish a modular framework for post-disaster AI assistance.

Future Work.

- 1) **Resolution unlocking during inference:** Evaluate with unbounded image resolution on free-tier higher-VRAM GPUs (e.g., Colab L4 or Kaggle P100) to quantify Deviation D4’s exact contribution to the accuracy gap, or feed VRT-upscaled images to the VLM as a preprocessing countermeasure.
- 2) **Textual bridge prompting:** Inject U-Net spatial findings (building counts, damage class distributions, quadrant locations) as structured text into the VLM prompt rather than feeding image crops. This lets the VLM reason over precise spatial data without relying on its vision encoder to resolve fine-grained coordinates.
- 3) **Survivability heatmap generation:** Apply `cv2.distanceTransform` or 2D Kernel Density Estimation over U-Net damage masks to produce continuous spatial gradients indicating survivability zones, enabling field responders to visually identify safe assembly points and high-danger corridors.
- 4) **Sequence-level distillation (7B → 3B):** Generate chain-of-thought reasoning traces from the fine-tuned 7B model over the training set, then fine-tune Qwen2.5-VL-3B on the synthetic outputs for edge deployment on rescue laptops (~6 GB VRAM).
- 5) **DisasTeller backend swap:** Replace the Gemini Flash-Lite vision backend in DisasTeller with our fine-tuned Qwen2.5-VL, creating a fully autonomous end-to-end disaster response system using a domain-specific model instead of a generic proprietary API.
- 6) **Instance segmentation:** Migrate from semantic segmentation (U-Net) to Mask R-CNN so adjacent buildings can be separated and counted individually rather than merged via connected-component approximation.
- 7) **EdgeV-SE deployment:** EdgeV-SE [7] self-reflective fine-tuning with asymmetric dual pathways for deployment on NVIDIA Jetson Orin Nano edge devices in field conditions.
- 8) **Open-ended evaluation:** GPT-4.1-as-judge scoring of Disaster Caption and Restoration Advice tasks using the paper’s Appendix C.2 rubrics, if API budget becomes available.
- 9) **Full unified deployment:** Package the already-validated components, i.e., VRT super-resolution preprocessing, U-Net spatial extraction (implemented), fine-tuned VLM

reasoning (implemented), and DisasTeller multi-agent report generation via RAG into a single containerized deployable service with real-time drone feed ingestion and automated end-to-end inference.

ACKNOWLEDGMENT

The authors acknowledge the free GPU resources provided by Kaggle and Google Colab, and the DisasterM3 dataset released by Wang et al. [1] under CC BY-NC license.

REFERENCES

- [1] J. Wang, W. Xuan, H. Qi, Z. Liu, K. Liu, Y. Wu, H. Chen, J. Song, J. Xia, Z. Zheng, and N. Yokoya, “DisasterM3: A remote sensing vision-language dataset for disaster damage assessment and response,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2505.21089.
- [2] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang et al., “Qwen2.5-VL technical report,” *arXiv preprint arXiv:2502.13923*, 2025.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [7] Y. Jeon, S. Lee, and W. Kim, “EdgeV-SE: Self-reflective fine-tuning framework for edge-deployable vision-language models,” *Applied Sciences*, vol. 16, no. 2, 2026.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [9] R. Gupta, B. Goodman, N. Patel, R. Hosfelt, S. Sajeev, E. Heim, J. Doshi, K. Lucas, H. Choset, and M. Gaston, “Creating xBD: A dataset for assessing building damage from satellite imagery,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.
- [10] CrewAI, “CrewAI: A framework for orchestrating role-playing AI agents,” <https://github.com/crewAIInc/crewAI>, 2024.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [12] J. Liang, J. Cao, Y. Fan, K. Zhang, R. Ranjan, Y. Li, R. Timofte, and L. Van Gool, “VRT: A video restoration transformer,” *arXiv preprint arXiv:2201.12288*, 2022.
- [13] A. Al-Mohannadi, A. Firoz, Y. Yang, M. Imran, and F. Ofli, “DisasterVQA: A visual question answering benchmark dataset for disaster scenes,” in *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2026, arXiv:2601.13839.
- [14] Gemini Team, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [15] P. Iakubovskii, “Segmentation models pytorch,” https://github.com/qubvel/segmentation_models.pytorch, 2019.